

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - Database Design and Connectivity

Course Code:	DSA0501	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	AT1 – Database Design and Connectivity
Weightage :		Total Marks:	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)			
Student Name:	P.Sunayana	Reg. No.:	192424303
Section / Batch:	Slot-A 2024	Date of Submission:	29-07-2026
Faculty In charge:	K Veena devi	Project Mode:	Individual Submission

Code of Conduct

I _____ **P.Sunayana 192424303** _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____ **P.Sunayana** _____

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

CO2 – AT1 – Database Design and Connectivity Sample Questions

1. Student Database Design and Connectivity (10 Marks)

A college wants to automate the management of student records. Design an SQLite database containing a **Student** table with appropriate fields and a primary key. Write a Python program to connect to the SQLite database, create the table if it does not exist, insert at least five student records, and display all the records stored in the table.

Code:

```
import sqlite3
con = sqlite3.connect("student.db") cur =
con.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Student( id
INTEGER PRIMARY KEY,
name TEXT,
dept TEXT,
marks INTEGER) """)
students=[
(1,"Anu","CSE",90),
(2,"Rahul","IT",85),
(3,"Priya","ECE",88),
(4,"Kiran","EEE",82),
(5,"John","CSE",95)
]
cur.executemany("INSERT OR REPLACE INTO Student VALUES(?,?,?,?)",students) for i in
```

```
cur.execute("SELECT * FROM Student"):
    print(i)
con.commit()
con.close()
```

Output:

```
(1, 'Anu', 'CSE', 90)
(2, 'Rahul', 'IT', 85)
(3, 'Priya', 'ECE', 88)
(4, 'Kiran', 'EEE', 82)
(5, 'John', 'CSE', 95)
```

2. Library Management Database (10 Marks)

A library plans to maintain information about its books using an SQLite database. Design a **Book** table containing suitable attributes such as Book ID, Title, Author, Publisher, and Price. Write a Python program to establish a connection with the database, create the table, insert sample records, update the price of a selected book, and retrieve the updated information.

Code:

```
import sqlite3
con=sqlite3.connect("library.db")
cur=con.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Book( id
INTEGER PRIMARY KEY,
title TEXT,
author TEXT, publisher TEXT, price REAL) """)
books=[ (1,"Python","ABC","Pearson",500),
(2,"Java","XYZ","McGraw",650),
(3,"DBMS","Ram","Oxford",700)
]
cur.executemany("INSERT OR REPLACE INTO Book VALUES(?,?,?,?)",books)
cur.execute("UPDATE Book SET price=750 WHERE id=2")
```

```

for i in cur.execute("SELECT * FROM Book WHERE id=2"): print(i)
con.commit()
con.close()

```

Output:

```

(2, 'Java', 'XYZ', 'McGraw', 750.0)

```

3. Employee and Department Database Design (10 Marks)

A company wants to store employee information and department details in an SQLite database. Design two related tables, **Department** and **Employee**, by identifying appropriate primary and foreign keys. Write a Python program to connect to the database, create both tables, insert sample data into each table, and display the employee name along with the corresponding department name using an SQL JOIN query.

Code:

```

import sqlite3.

connect("company.db") cur=con.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS Department(deptid INTEGER PRIMARY
KEY,dname TEXT)")

cur.execute("CREATE TABLE IF NOT EXISTS Employee(empid INTEGER PRIMARY
KEY,name TEXT,deptid INTEGER,FOREIGN KEY(deptid) REFERENCES
Department(deptid))")
cur.executemany("INSERT OR REPLACE INTO Department VALUES(?,?)",
[(1,"HR"),(2,"Sales"),(3,"IT")])
cur.executemany("INSERT OR REPLACE INTO Employee VALUES(?,?,?)",
[(1,"Anu",1),(2,"Rahul",2),(3,"John",3)])
for i in cur.execute("""
SELECT Employee.name,Department.dname FROM
Employee
JOIN Department
ON Employee.deptid=Department.deptid """):
    print(i)

```

```
con.commit()
```

```
con.close()
```

Output:

```
('Anu', 'HR')
('Rahul', 'Sales')
('John', 'IT')
```

4. Hospital Patient Database (10 Marks)

A hospital requires a database to manage patient information. Design an SQLite database with a **Patient** table containing suitable fields and constraints. Write a Python program to connect to the database, create the table, insert patient records, update the contact number of a specified patient, delete a discharged patient's record, and display the remaining records.

Code:

```
import sqlite3
con=sqlite3.connect("hospital.db")
cur=con.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Patient( id
INTEGER PRIMARY KEY,
name TEXT,
disease TEXT,
contact TEXT) """)
patients=[
(1,"Ravi","Fever","9876543210"),
(2,"Anu","Cold","9876543211"),

(3,"John","Diabetes","9876543212")
]
cur.executemany("INSERT OR REPLACE INTO Patient VALUES(?,?,?,?)",patients)
cur.execute("UPDATE Patient SET contact='9999999999' WHERE id=2") cur.execute("DELETE
```

```

FROM Patient WHERE id=1")
for i in cur.execute("SELECT * FROM Patient"): print(i)
con.commit()
con.close()

```

Output:

```

(2, 'Anu', 'Cold', '9999999999')
(3, 'John', 'Diabetes', '9876543212')

```

5. Online Course Registration System (10 Marks)

An online learning platform needs to maintain information about students and the courses they have registered for. Design two related SQLite tables, **Student** and **Course_Registration**, using appropriate primary and foreign keys. Write a Python program to connect to the database, create the tables, insert sample records, and retrieve the names of students along with the courses they have registered for using an SQL JOIN operation.

Code:

```

import sqlite3
con=sqlite3.connect("course.db")
cur=con.cursor()

cur.execute("CREATE TABLE IF NOT EXISTS Student(id INTEGER PRIMARY KEY,name TEXT)")

cur.execute("CREATE TABLE IF NOT EXISTS Course_Registration(id INTEGER PRIMARY KEY,course TEXT,studentid INTEGER,FOREIGN KEY(studentid) REFERENCES Student(id))")
cur.executemany("INSERT OR REPLACE INTO Student VALUES(?,?)", [(1,"Anu"),(2,"Rahul")])
cur.executemany("INSERT OR REPLACE INTO Course_Registration VALUES(?,?,?)",

[(1,"Python",1),(2,"Java",2)])
for i in cur.execute("""
SELECT Student.name,Course_Registration.course FROM Student
JOIN Course_Registration
ON Student.id=Course_Registration.studentid """):

```

```
print(i)
con.commit()
con.close()
```

Output:

```
('Anu', 'Python')
('Rahul', 'Java')
```

6. Banking Account Management System (10 Marks)

A bank wants to maintain customer account information using an SQLite database. Design an appropriate database containing an **Account** table with suitable attributes and constraints.

Write a Python program to connect to the database, create the table, insert customer account details, update the account balance of a specified customer, and display all customer records.

Code:

```
import sqlite3
con=sqlite3.connect("bank.db")
cur=con.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Account( accno
INTEGER PRIMARY KEY,
name TEXT,
balance REAL) """)
cur.executemany("INSERT OR REPLACE INTO Account VALUES(?,?,?)", [(101,"Anu",10000),
(102,"Rahul",15000),
(103,"John",12000)])
cur.execute("UPDATE Account SET balance=18000 WHERE accno=102")
cur.execute("SELECT * FROM Account"):
    print(i)
con.commit()
con.close()
```

Output:

```
(101, 'Anu', 10000.0)
(102, 'Rahul', 18000.0)
(103, 'John', 12000.0)
```

7. Inventory Management System (10 Marks)

A retail store needs to maintain product inventory using SQLite. Design a **Product** table containing Product ID, Product Name, Category, Quantity, and Unit Price. Write a Python program to create the database, insert sample products, identify products with quantity less than 10, update their stock quantity, and display the updated inventory.

Code:

```
import sqlite3
con=sqlite3.connect("inventory.db")
cur=con.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS Product( id
INTEGER PRIMARY KEY,
name TEXT,
category TEXT, qty
INTEGER,
price REAL) """)
products=[ (1,"Mouse","Electronics",5,500),
(2,"Keyboard","Electronics",20,800),
(3,"Pen","Stationery",8,20)
]
cur.executemany("INSERT OR REPLACE INTO Product VALUES(?,?,?,?,?)",products)
cur.execute("UPDATE Product SET qty=25 WHERE qty<10")
```



```

for i in cur.execute("SELECT * FROM Product"): print(i)
con.commit()
con.close()

```

Output:

```

(1, 'Mouse', 'Electronics', 25, 500.0)
(2, 'Keyboard', 'Electronics', 20, 800.0)
(3, 'Pen', 'Stationery', 25, 20.0)

```

8. Course and Faculty Management System (10 Marks)

A university wants to maintain details of faculty members and the courses they teach. Design two related tables, **Faculty** and **Course**, using appropriate primary and foreign keys. Write a Python program to create the database, insert sample records, and display each faculty member along with the courses assigned using an SQL JOIN query.

Code:

```

import sqlite3
con=sqlite3.connect("faculty.db")
cur=con.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS Faculty(id INTEGER PRIMARY KEY,name TEXT)")

cur.execute("CREATE TABLE IF NOT EXISTS Course(id INTEGER PRIMARY KEY,cname TEXT,fid INTEGER,FOREIGN KEY(fid) REFERENCES Faculty(id))")

cur.executemany("INSERT OR REPLACE INTO Faculty VALUES(?,?)",
[(1,"Dr.Raj"),(2,"Dr.Kumar")])

cur.executemany("INSERT OR REPLACE INTO Course VALUES(?,?,?)",
[(1,"Python",1),(2,"DBMS",2)])

for i in cur.execute("""
SELECT Faculty.name,Course.cname FROM
Faculty

```

JOIN Course

```
ON Faculty.id=Course.fid """):
```

```
    print(i)
```

```
con.commit()
```

```
con.close()
```

Output:

```
( 'Dr.Raj', 'Python' )
( 'Dr.Kumar', 'DBMS' )
```

9. Vehicle Registration Database (10 Marks)

A transport department plans to maintain vehicle registration details in an SQLite database.

Design a **Vehicle** table with suitable attributes and constraints. Write a Python program to connect to the database, create the table, insert vehicle records, search for a vehicle using its registration number, and display the retrieved information.

Code:

```
import sqlite3
```

```
con=sqlite3.connect("vehicle.db")
```

```
cur=con.cursor()
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Vehicle( regno
```

```
TEXT PRIMARY KEY,
```

```
owner TEXT,
```

```
model TEXT, year
```

```
INTEGER) """)
```

```
vehicles=[ ("TN01AB1234","Ravi","Swift",2022),
```

```
("TN10CD5678","Anu","i20",2021)
```

```
]
```

```
cur.executemany("INSERT OR REPLACE INTO Vehicle VALUES(?,?,?,?)",vehicles)
```

```
for i in cur.execute("SELECT * FROM Vehicle WHERE regno='TN01AB1234'"): print(i)
```

```
con.commit()
```

```
con.close()
```

Output:

```
('TN01AB1234', 'Ravi', 'Swift', 2022)
```

10. Hospital Appointment Management System (10 Marks)

A hospital maintains separate tables for **Doctors** and **Appointments**. Design both tables using appropriate primary and foreign keys. Write a Python program to create the tables, insert sample records, and display the doctor name along with the appointment details using a JOIN query.

Code:

```
import sqlite3
con=sqlite3.connect("appointment.db")
cur=con.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS Doctor(id INTEGER PRIMARY KEY,name
TEXT)")

cur.execute("CREATE TABLE IF NOT EXISTS Appointment(id INTEGER PRIMARY
KEY,patient TEXT,date TEXT,doctorid INTEGER,FOREIGN KEY(doctorid)
REFERENCES Doctor(id))")
cur.executemany("INSERT OR REPLACE INTO Doctor VALUES(?,?)",
[(1,"Dr.Ram"),(2,"Dr.Sita")])
cur.executemany("INSERT OR REPLACE INTO Appointment VALUES(?,?,?,?)",
[(1,"Anu","2026-07-29",1),
(2,"Rahul","2026-07-30",2)])
for i in cur.execute("""
SELECT Doctor.name,Appointment.patient,Appointment.date
FROM Doctor JOIN
Appointment
ON Doctor.id=Appointment.doctorid """):
    print(i)
```

```
con.commit()
```

```
con.close()
```

Output:

```
· ('Dr.Ram', 'Anu', '2026-07-29')  
  ('Dr.Sita', 'Rahul', '2026-07-30')
```